LAB 3

Applied introduction to time series, NNs and evaluation metrics, CUDA & Google Colab

Time Series

A type of temporal data where observations are ordered in time, often at regular intervals.

Examples: sensor readings, stock prices, heart rate.

Time series is different from what you've seen so far in that observations are not independent, that is: order matters.

Often contains trends, seasonality, autocorrelation.

Typical ML tasks include forecasting, classification, anomaly detection.

Classical ML models are *not time-aware,* time is encoded ‘manually’ through feature extraction.

Time Series

Neural networks (e.g. RNNs, LSTMs, Transformers) can process entire sequences and the order of observations is part of the input.

The model can learn short- and long-term dependencies “automatically”.

So:

‘Classical’ ML:
“Predict output from these features!”

NN:s:
“learn how the past influences the future!”

Shapes in Python

Data is typically held as **NumPy arrays / tensors**

Their shape describes their dimensions:

Common examples:

(T,) single waveform / time series

(T, C) multi-channel signal (e.g. stereo, sensors)

(N, T) batch of time series

(N, T, C) batched, multi-channel sequences

shape() method in Python to get a sense of your data. Certain models require certain shapes.

NN preparations

The same cleaning principles apply for data trained with NN as before.

For time series we need to decide window size, step size and perhaps more.

For images (CNN) we need to reshape data into tensors, you can use OpenCV:
https://pypi.org/project/opencv-python/

Designing NNs

What you need to decide on and prefferably justify:

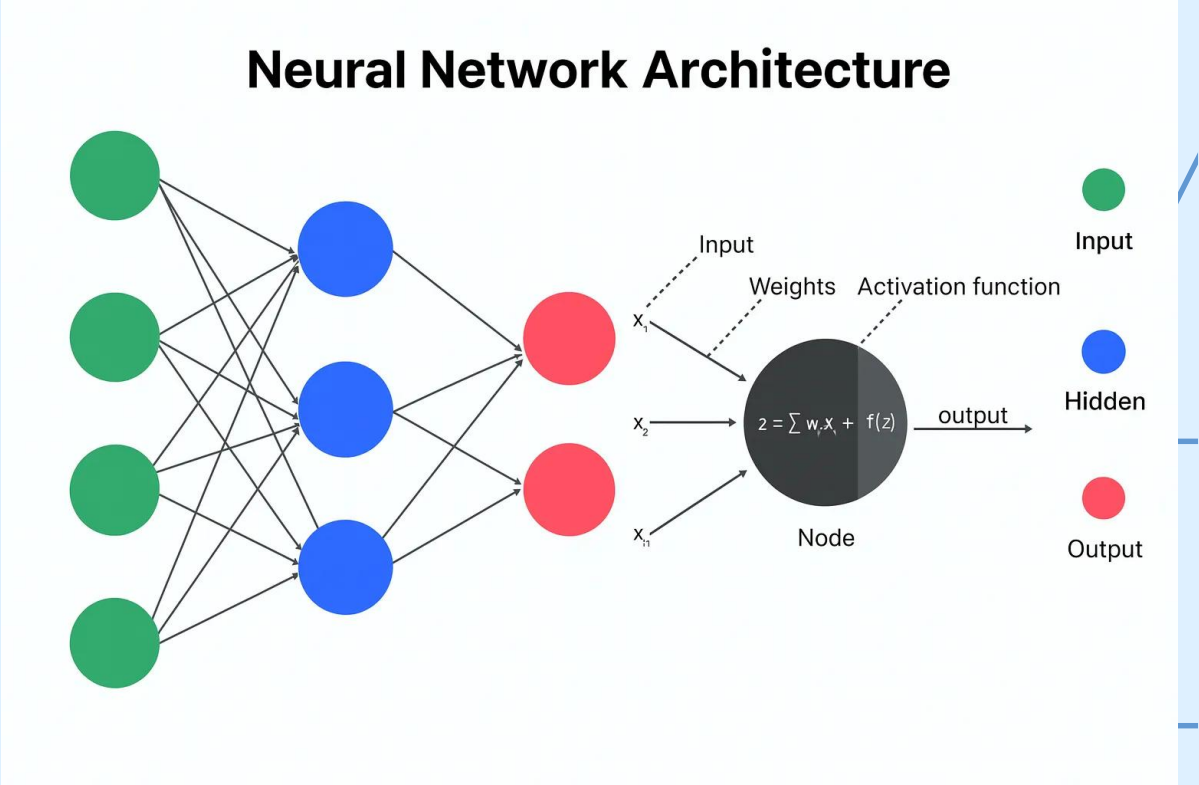
Type: What type of NN is suitable for my use-case? Why?

Hyperparameters: Learning rate, Batch size, Optimizer, Loss function...

Activation Functions: ReLU, Sigmoid, Tanh, Softmax...

Layers: Input layer, Hidden layers, Output layer, Normalization layers, Dropout...

Epochs: Number of passes through training data, early stopping...



Activation Functions

- Activation functions decide **whether a neuron is activated or not**.
- They add **non-linearity** to the model so it can learn complex patterns.
- Without activation functions, a neural network would behave like a **linear model**, no matter how many layers it has.

Layers

- Layers are the **building blocks** of a neural network.
- Each layer processes information and passes it to the next one.
- A neural network is made of several stacked layers.

Loss Function

- A loss function is the mathematical penalty the neural network minimizes during training to adjust its weights so that predictions move closer to the true values.
- Loss tells the model how to learn; metrics tell you how well it learned.

Why loss matters?

- Detecting overfitting
- Early stopping
- Hyperparameter tuning
- Monitoring convergence
- Comparing models under same scaling

Epochs

- An epoch means one full pass of the entire training dataset through the model.
- During one epoch, the model sees all training samples once.
- Training usually needs many epochs to learn well.

Why We Use Epochs?

- The model does not learn everything in one pass.
- Each epoch helps the model **improve its weights**.
- More epochs → better learning (up to a point).

NN Libraries

PyTorch

TensorFlow

Keras

CUDA

CUDA is NVIDIA's parallel computing platform that enables GPU for computing tasks.

To enable your local GPU for training models:

- **install the appropriate drivers, CUDA, and cuDNN libraries**

https://docs.nvidia.com/cuda/cuda-installation-guide-microsoft-windows/contents.html

- **ensure that TensorFlow or PyTorch, is set up to utilize the GPU in code**.

Google Colab

To speed up training you can also set up Google Colab linked to your Google account, through that you can access their GPU:s remotely.

Enough for 'basic' NN:s, could still take long for LLM:s.

More on evaluation metrics

No single metric tells the whole story.

Thought experiment: Fire alarm

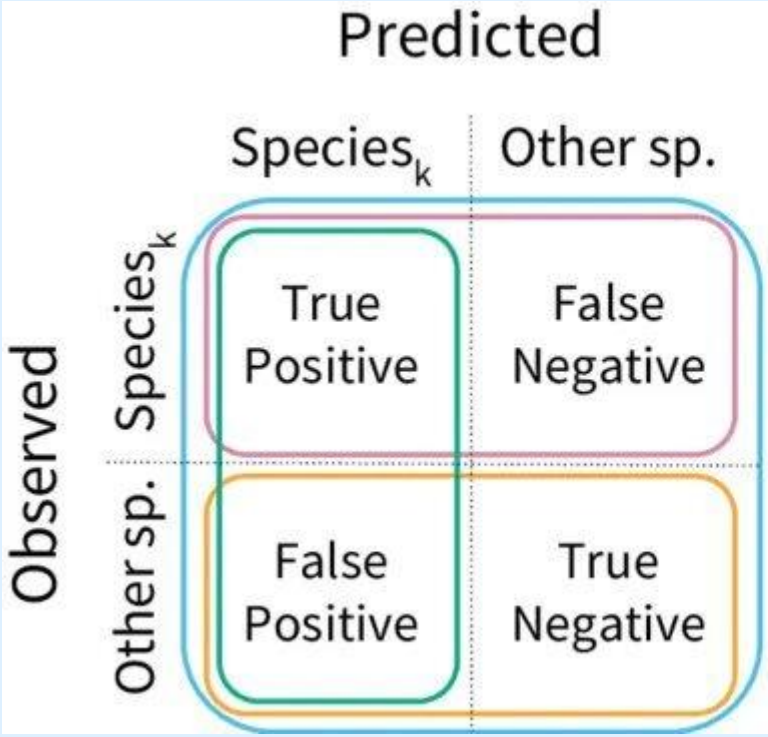
Potential metrics for classification tasks: Accuracy, Recall, Precision, F1

(for regression: MSE, MAE, RMSE, R2...)

You should twist and turn, think, argue!

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

The formulas



$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Specificity} = \frac{TN}{TN + FP}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

ROC - Receiver Operating Characteristic
AUC - Area Under the Curve

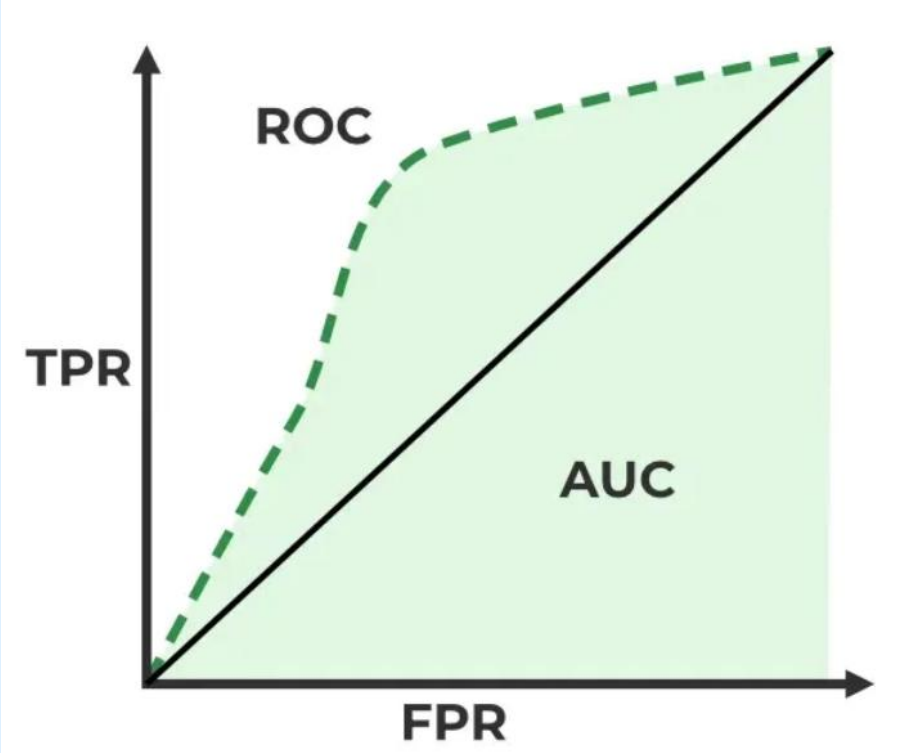
The ROC curve shows how well a classifier separates two classes

Y-axis: True Positive Rate (how many positives are correctly detected)

X-axis: False Positive Rate (how many false alarms occur)

Each point on the curve represents a different **decision threshold**

AUC (Area Under the (ROC) Curve) measures the overall performance of a classifier. Values range from 0 to 1.



For the submission

Again, see the submission_example folder on Canvas (the README.md)

Please submit the notebooks with included output (you run through it before submitting).

It’s time to explore!

- Download the LAB 3 folder on Canvas.
- Run the notebooks (lab3_ts → lab3_nn → lab3_timeseries_nn)
- Understand what the codes do.

- … and/or set up CUDA, Google Colab, explore Pytorch/TF/Keras